

ReflectAI — Presentation Speech

Phase-2 Review-1 · Full speaking script, written in simple English

Presenters: Ajay M, Adarsha P, Anjan Gowda, Divit B S | Guide: Dr Vidya Y

Slide 1 — Title Slide

Good morning everyone. My name is Ajay M. My team is Adarsha P, Anjan Gowda, and Divit B S. Our guide is Dr Vidya Y.

Today we are presenting our major project. It is called ReflectAI. The full title is: ReflectAI, a Depth-Adaptive Socratic Reflection Engine for Longitudinal AI Journaling.

In simple words, it is a smart journaling app. It asks you good questions. It helps you understand your own feelings over time.

This is our Phase 2, Review 1. We will show you what we built, why we built it this way, and a working demo.

Slide 2 — Agenda

Here is our plan for today.

First, we will explain the idea behind ReflectAI. Then we will show the research that supports our idea.

After that, we will explain the problem we are solving, and our main goals.

Next comes the big part: our system architecture. Here we will also explain why we chose each technology, and why we did not choose other options.

Then we will show live results. At least 60 to 70 percent of the work is already done and working.

After that, we will show our timeline, our future plans, and where we stand with publishing our work. Let's begin.

Slide 3 — Introduction

Let's start with why we built this. Journaling, or writing about your feelings, is proven to help people feel better and understand themselves. But there is a big problem: most people stop journaling after just a few days. Why? Because there is no one asking them questions, no structure, and no feedback.

So we built ReflectAI. Inside the app, it is branded as "Equoria." It is a full digital health platform. It lets you write journal entries freely. But it also gives you an AI partner. This AI does not just listen — it asks follow-up questions, like a good therapist or a good friend would. We call this

"Socratic questioning." That simply means asking simple, thoughtful questions that make you think deeper, instead of giving you answers directly.

Here is how it works. Every time you write an entry, the system tags it with your mood and the time. Then our AI reads your past entries and asks a smart follow-up question based on your own history. If the AI service is down for some reason, we do not want the app to break. So we built a backup system — a simpler, rule-based system that still asks good questions, without needing the AI. This means the app always works, even if something goes wrong with the AI connection.

We also built a companion app for iPhone. It reads health data from Apple Health — like your sleep, steps, heart rate, and stress level. This data is sent to our platform. Then, in a feature we call "Retrospect," we compare your mood with your health data over time. For example, we can show you: on days you sleep less, your mood tends to be lower.

On the engineering side: our website is built with React and Vite. Our backend server uses Node.js, Express, and MongoDB. Our iPhone app uses SwiftUI and HealthKit. And security was built in from day one — not added later. This includes secure logins, two-factor authentication, encrypted data, and limits on how many requests someone can make, to stop attacks.

For this review, we are showing a real, working system. Not just ideas or mockups. Login, journaling, AI-guided reflection, mood and health analytics, Apple Health syncing, and a fully customizable look — all of it works today.

Slide 4 — Literature Survey

Before we built anything, we studied what research already says. Here is what we found.

First, many studies from 2023 to 2026 show that AI chatbots can genuinely help reduce symptoms of depression and anxiety. And AI journaling is being recognized as its own new way of using AI, different from just chatting with a bot.

Second, newer studies from 2024 and 2025 show that AI models can successfully do Socratic questioning — the same style of questions used in real therapy, called CBT. Mental health professionals checked real conversations and confirmed the AI was doing this correctly. This directly shaped how we built our own follow-up question engine.

Third, research shows that simple data from your phone or watch — like your steps, sleep, and movement — can actually predict your mood and stress. This is why we built the health-mood correlation feature.

Fourth, there is older, very solid research, going back decades, that just writing about your feelings, even without AI, improves both your mental and physical health. This is the foundation our whole app stands on.

Fifth, modern AI text models can detect the mood of a piece of writing with about 90 percent accuracy. This supports our mood-tagging feature.

Sixth, and this is important: a 2024 study found that many mental health apps have serious security problems. One study found over 1,500 security issues across just 10 apps. This is exactly why we made security a top priority from the very beginning, and why we built our own testing system to check for weaknesses.

So, to summarize: research proves that journaling helps, AI can do therapy-style questioning well, and health data can predict mood. But almost no app combines all three, with strong security. That is the gap. That is where ReflectAI fits in.

Slide 5 — Problem Statement

Now let's talk about the actual problem.

Most journaling apps today are just empty notebooks. You write, and nothing happens back. No feedback. On the other side, most AI mental health chatbots have the opposite problem — they do not remember you. Every conversation starts from zero. They do not know your writing history. They do not know your health.

Because of this, users are left completely on their own. Nobody is helping them notice patterns, like: you always feel stressed after a bad night's sleep. There is no safe, always-available way to go deeper into your feelings, past just a short, surface-level entry. The result: people write shallow entries, they stop journaling quickly, and early warning signs of stress or burnout get missed.

So, in one sentence: there is no single, secure platform that brings together guided journaling, AI-driven follow-up questions, and real health data, all working together as one continuous feedback loop. ReflectAI is our answer to that gap.

Slide 6 — Objectives

Based on that problem, here is what we set out to build. Five main goals.

One: build a secure, complete journaling platform, where every entry has a mood tag, and can optionally be linked to your health data.

Two: build an AI reflection engine. It should ask smart, personal follow-up questions based on your own journal history. And it must have a safe backup plan, so the experience never completely breaks, even without AI.

Three: automatically find patterns for the user — repeating themes, behavior loops, and connections between mood and health — instead of making the user find these patterns by themselves.

Four: connect to real health data — steps, sleep, heart rate, stress — through a native iPhone app using Apple HealthKit, syncing in near real time in the background.

Five: build the whole platform to a professional security standard. This means secure login tokens, two-factor authentication, rate limiting to stop abuse, data encryption, and our own live security-testing tool.

Slide 7 — Methodology / System Architecture

Now for the heart of the talk: how the system is actually built, and why we made each choice.

Let's walk through the diagram from top to bottom, the way data actually flows.

At the top, we have three entry points. First, the client — this is our website, built with React 18 and Vite. It has the dashboard, the journal, the chat, the retrospect analytics, the health page, and settings, plus nice touches like a 3D "Memory Globe" and smooth animations. Second, the iOS companion app, built with SwiftUI, which talks to Apple HealthKit to read your health data in the background. Third, the AI Reflection Engine, which uses OpenAI's API to generate smart, Socratic-style questions, with a safe fallback for when there is no AI connection.

Below that is the backend, built with Node.js and Express. This is the "brain" that receives requests from the website and the iPhone app, checks that they are allowed using secure login tokens, validates the data, and then talks to the database.

Below that is the database: MongoDB, with a tool called Mongoose on top of it, which enforces structure and validation rules. This stores users, journal entries, chat history, health data, and analysis results. Sensitive fields are encrypted.

And at the very bottom is a security and reliability layer that runs underneath everything — things like blocking repeated failed logins, two-factor authentication, encrypted data, and a testing tool we built ourselves to try to break into our own app and find weaknesses before anyone else does.

Now, the important part: why did we choose these specific technologies, and not others?

WHY REACT AND VITE FOR THE WEBSITE, AND NOT ANGULAR OR VUE? React has the biggest ecosystem of ready-made tools. We needed animation (Framer Motion), charts (Recharts), and even 3D graphics (Three.js, for our Memory Globe). All of these work best, and are best documented, with React. Angular is much heavier and more rigid. It forces a strict structure that would have slowed us down, since our four-person team needed to move fast and change designs often over 16 weeks. Vue and Svelte are lighter, but they have smaller communities and fewer ready-made libraries for the specific things we needed, like 3D visualization. Vite, specifically, gives us very fast reload times while coding, much faster than older tools like Create React App, which mattered a lot given our tight timeline.

WHY NODE.JS AND EXPRESS FOR THE BACKEND, AND NOT PYTHON, JAVA, OR GO? Our actual "smart" work, the AI questioning, is not done by our own server. It is done by OpenAI's API. So our backend's real job is mostly quick back-and-forth: talk to the database, talk to the AI service, send small pieces of data back and forth. Node.js is built exactly for this kind of work: lots of small, fast, ongoing conversations, not heavy number-crunching. Also, since our frontend is already in JavaScript, using Node.js means our whole team writes in one language across the

entire app, with no need to switch between languages. Java's Spring Boot is powerful, but it is heavier and slower to build with for a small student team on a 16-week deadline. Python's Django or Flask would have been fine for AI logic, but our AI logic lives inside OpenAI's API, not in our own code, so Python's data-science strengths were not actually needed here. Go is fast, but it has a much smaller set of ready-made packages for things we needed quickly, like validation libraries and security middleware.

WHY MONGODB, AND NOT A TRADITIONAL DATABASE LIKE POSTGRESQL OR MYSQL? Our data does not look the same for every entry. A journal entry might have three mood tags, or none. A chat session has a growing list of messages of different shapes. Health data has different fields depending on the day. MongoDB stores data as flexible documents, which fit this kind of changing data naturally, without redesigning database tables every time we added a new feature. A database like PostgreSQL would need many separate tables, linked together, for a journal entry, its mood tags, and its linked health data. This adds complexity we did not need. Since our data model kept evolving fast during weeks 4 to 11 of the project, Mongoose gave us the best of both worlds: MongoDB's flexibility, plus schema rules and validation on top, so data still stays clean and consistent.

WHY OPENAI'S API FOR THE AI ENGINE, AND NOT OUR OWN AI MODEL, OR A SMALLER MODEL RUNNING ON OUR OWN SERVERS? Building or training our own AI model good enough for real Socratic-style questioning is a massive task, far beyond the scope of a 16-week student project. The research we reviewed shows this kind of careful questioning needs a genuinely strong AI model to be safe and effective. Smaller, self-hosted open models are not yet proven at the same level for this specific job, and hosting our own AI servers would have added huge cost and complexity for a project like ours. That said, we know that depending fully on one outside AI service is a risk. What if the connection fails, or the API key is missing? That is exactly why we built the heuristic fallback: a rule-based backup system that still asks good, non-repeating follow-up questions, without needing any AI connection at all. So the app never fully breaks. In the future, as listed in our future scope, we want to explore on-device AI to reduce this dependency further.

WHY NATIVE SWIFTUI FOR THE IPHONE APP, INSTEAD OF A CROSS-PLATFORM TOOL LIKE REACT NATIVE OR FLUTTER THAT COULD COVER BOTH IPHONE AND ANDROID AT ONCE? Apple HealthKit, the tool that reads your health data, works best and most reliably with native Apple code. Background syncing, getting your health data updated automatically even when the app is not open, is especially tricky and less reliable when going through a cross-platform bridge like React Native or Flutter. Since accurate, reliable health data was a core requirement, not a nice-to-have, we chose to build natively in SwiftUI first, to get this right on iPhone. We do plan to build an Android version later, using Android's own Health Connect tool, which is listed in our future scope.

WHY DID WE ADD SECURITY THIS EARLY, INSTEAD OF ADDING IT LATER? We found real research showing that mental health apps are commonly under-secured. One study found over 1,500 security problems across just 10 apps. Given that our app deals with people's private thoughts and health data, we could not treat security as an afterthought. So instead of building first and securing later, we engineered secure logins, two-factor authentication, data encryption, and rate

limiting into the system from the start, and we built our own testing tool to actively try to break in and find weaknesses.

So overall, every technology choice here was made for one of two reasons. Either it was the best fit for the specific problem, like MongoDB for flexible data, or native Swift for reliable HealthKit access. Or it let a small, four-person student team move fast and safely within a 16-week timeline, like React and Node.js, where our whole team could work in one shared language.

Slide 8 — Results I: Journaling & AI Reflection

Now let's look at the actual working app. This is not a mockup. This is our real, running system.

Here is the dashboard. It shows a mood calendar, your journaling streak, and a short summary of your recent emotional pattern, generated automatically.

Here is the New Journal Entry screen. You write freely, and you can tag your mood — happy, calm, reflective, sad, stressed, or angry. The app also shows a quick health snapshot and a writing prompt to help you get started if you are stuck.

Here is the AI Reflection Chat. This is the core feature. You write something, and instead of just responding, the AI asks a follow-up question, grounded in your own words. For example, if you say you feel overwhelmed, it might ask what part of your day is taking the most emotional energy. This is the Socratic style we talked about earlier — leading you to your own insight, not just giving you a generic answer.

And here is Settings, where users can customize the whole look of the app. Dark or light theme, and even pick their own accent colors live, with sliders.

Slide 9 — Results II: Analytics & Health Sync

Next, our analytics features.

This is Retrospect. It shows an emotional timeline, a chart of your mood intensity over recent days, plus recurring themes it found in your writing, like work, sleep, or relationships. It also shows behavior patterns it noticed, like: high workload connects to reduced breaks and higher stress. It ends with a smart follow-up question to help you reflect further.

This is our Apple Health sync page. It shows your weekly trends for steps, stress, and sleep, pulled directly from your iPhone in the background.

This is the Memory Globe, a fun 3D visual way to explore your journal entries. Each dot is a day you journaled, and you can drag it around and click on entries to revisit them.

And finally, the Health dashboard, showing today's numbers, like steps, heart rate, sleep, and stress score, plus your monthly averages.

Slide 10 — Timeline / Plan

Here is how we planned and tracked our 16-week project.

Weeks 1 to 3: planning and design — requirements, our literature research, the system architecture, and database design. Done.

Weeks 4 to 7: the core platform — secure login, two-factor authentication, journal creation, mood tagging, the dashboard, and our database models. Done.

Weeks 8 and 9: the AI reflection engine — connecting to OpenAI, building the fallback system, and making sure the AI does not repeat itself. Done.

Weeks 10 and 11: analytics and health syncing — the Retrospect feature, theme detection, the iPhone HealthKit app, and QR code pairing to connect the phone to your account. Done.

Week 12: security hardening — rate limiting, encryption, our security testing tool, and fixing six major risks we found. Done.

Week 13, which is right now: UI and UX polish — the Memory Globe, custom themes, layout improvements, and live quality testing. This is about 65 percent done.

Weeks 14 to 16: final testing, deployment, extended analytics, and preparing for our final demo. This is still planned ahead.

Slide 11 — Future Scope

Looking ahead, here is what we want to build next.

A new chart type, called a Nightingale Rose Chart, to show mood and emotion in a richer way across seasons. We are currently discussing the design.

An Android version of our companion app, using Android's Health Connect tool, to match what we have already built for iPhone.

Deeper emotion detection using more advanced AI language models, replacing our current simpler keyword-based system.

Support for images, voice notes, and videos in journal entries, not just text.

On-device AI processing, to reduce how much we depend on an outside AI service, and to make privacy even stronger.

Support for multiple languages, for both journaling and AI reflection.

And smarter, personalized habit reminders, building on our existing streak feature, based on the health-pattern research we studied earlier.

Slide 12 — Patent / Publication Details

On patents and publications: at this stage, we have not filed any patents, and we have not published anything yet. This is expected, since we are only at Phase 2.

However, we have submitted a paper about our Socratic reflection engine and our mood-health correlation method to the NMIT conference, called NMITCON 2026. We have received our invitation, and the conference is on 23rd September 2026.

We are also exploring a possible patent for one specific piece: our heuristic fallback system — the backup system that keeps AI-style reflection working safely and without repeating itself, even without a live AI connection. We believe this specific mechanism may be novel enough to protect.

Slide 13 — References & Closing

Finally, our references. We built this project on top of 15 solid academic sources, covering AI chatbots for mental health, Socratic and CBT-style questioning, digital health prediction from phone and wearable data, the original research on expressive writing, mood detection using AI language models, and security research on mental health apps. Full citations are listed on this slide for anyone who wants to look deeper.

To close: ReflectAI brings together three things that research says work — guided journaling, AI-driven Socratic questioning, and real health data — into one secure, working platform. We have shown you a real, running system today, not just a plan.

Thank you. We are happy to take your questions.